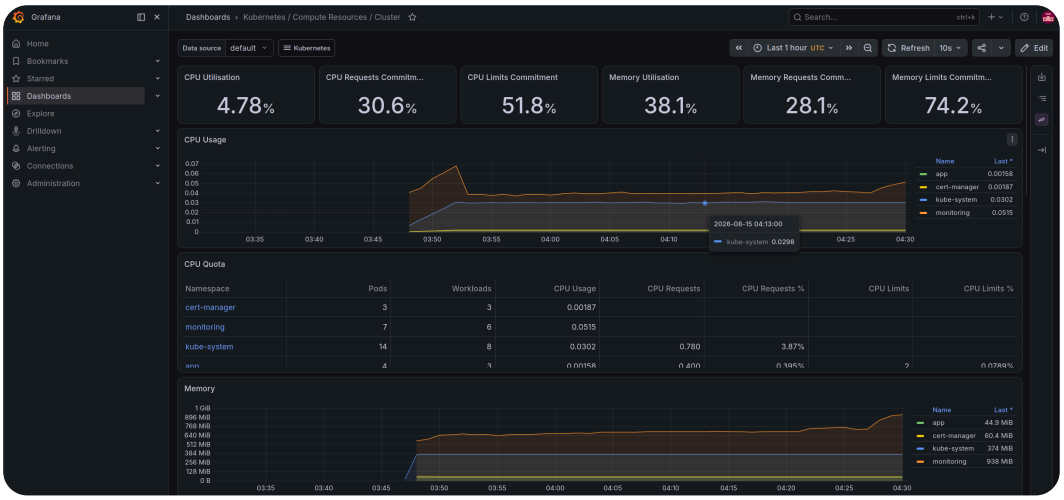
DEPLOY A PRODUCTION APP ON AWS EKS

By Ahmed Tetteh | August 2026



Building a Production-Grade Kubernetes Platform on AWS EKS

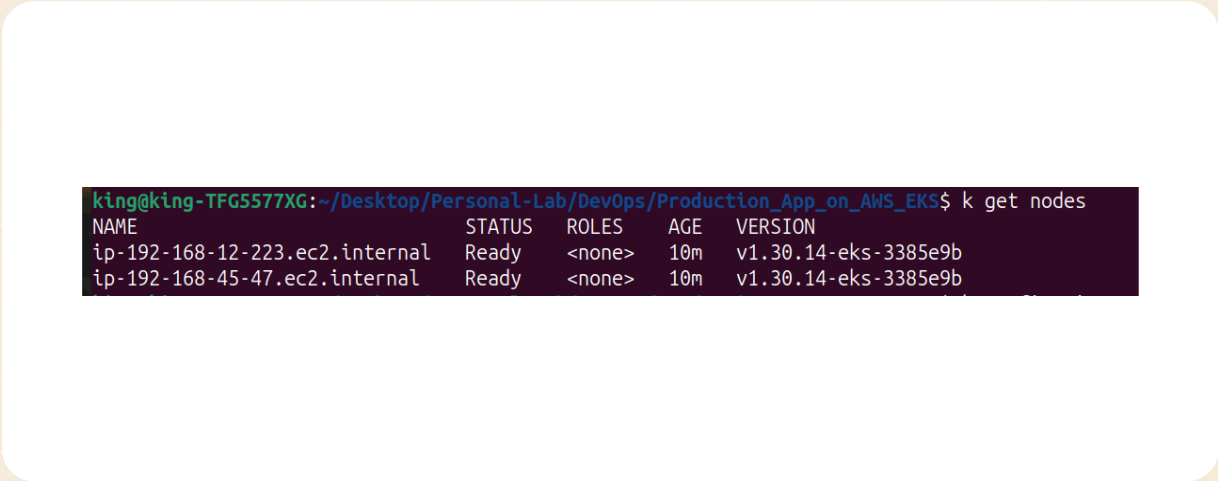
Project goals and architecture overview

In this project, I'm building a 2-node EKS cluster that deploys a 3-tier production level application with DNS and TLS baked within the architecture.

Provisioning the EKS Cluster

Cluster setup strategy

In this step, I'm setting up a 2-node eks cluster using the eksctl tool, and also set up an OIDC provider to allow AWS grants permissions to the pods on the nodes.



OIDC provider and IRSA integration

The OIDC provider allows pods and controllers within the cluster to have the exact permssions they need to communicate with other AWS services assume IAM roles.

Configuring DNS with Route53 and Porkbun

DNS delegation strategy

In this step, I'm setting up a Route53 hosted zone for a domain, so that anyone can reach the applications running in my cluster publicly.

```
;; ANSWER SECTION:
kahmedt.com.            172800  IN      NS      ns-1511.awsdns-60.org.
kahmedt.com.            172800  IN      NS      ns-127.awsdns-15.com.
kahmedt.com.            172800  IN      NS      ns-926.awsdns-51.net.
kahmedt.com.            172800  IN      NS      ns-2011.awsdns-59.co.uk.
```

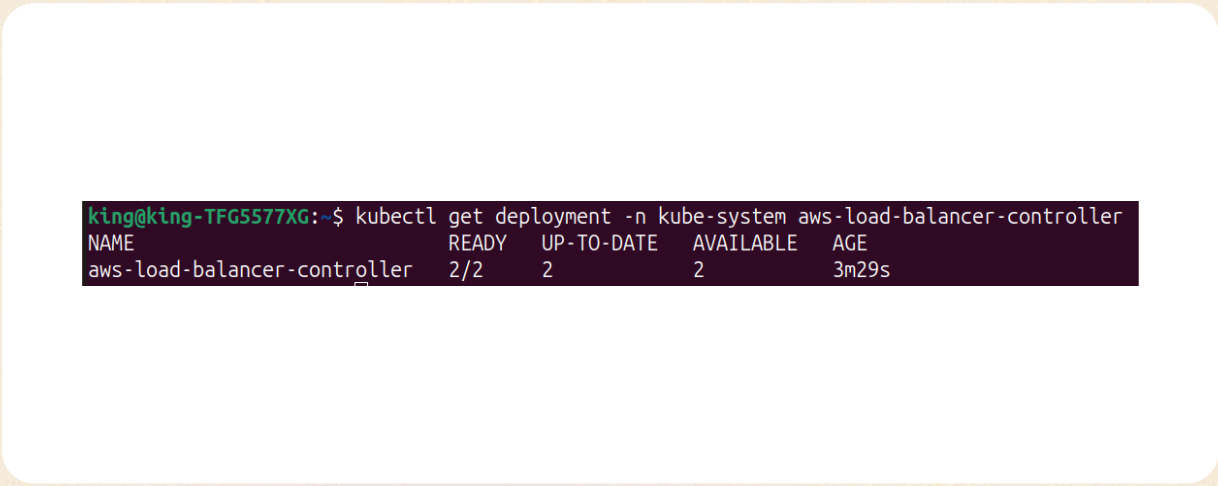
Why Route53 authority is required for automation

Route53 needs to be authoritative because it will serve as the single source of truth for all my DNS records relating to the domain name, and pass on the responsibility of managing the domain name to AWS. This makes it easier to validate TLS certificates later.

Installing the AWS Load Balancer Controller

ALB controller setup

In this step, I'm setting up an AWS Load Balancer Controller, so that I can route traffic to the pods in the cluster using Ingress resources.



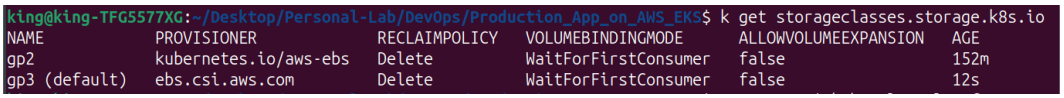
Least-privilege access with IRSA

IRSA provides least-privilege access by binding an IAM role to a specific Kubernetes service account, which means only the controller's using that service account inherit those spefic permissions attached to the role.

Enabling Persistent Storage with the EBS CSI Driver

EBS CSI driver installation

In this step, I'm setting up an EBS volume for persistant storage, so that data accommolated by the pods within my cluster can survive restarts.



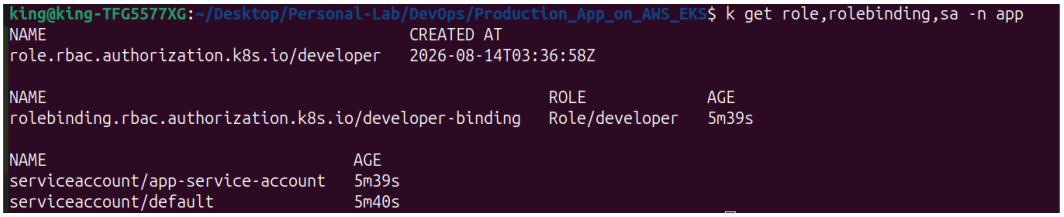
Volume binding mode explained

WaitForFirstConsumer is used because we want the EBS volume to only be provisioned when a pod requests it, and its been scheduled on a node.

Securing the Cluster with RBAC

Namespace and role configuration

In this step, I'm setting up RBAC(Role-Based Access Control), so that I can define fine-rained permissions for pods and users.



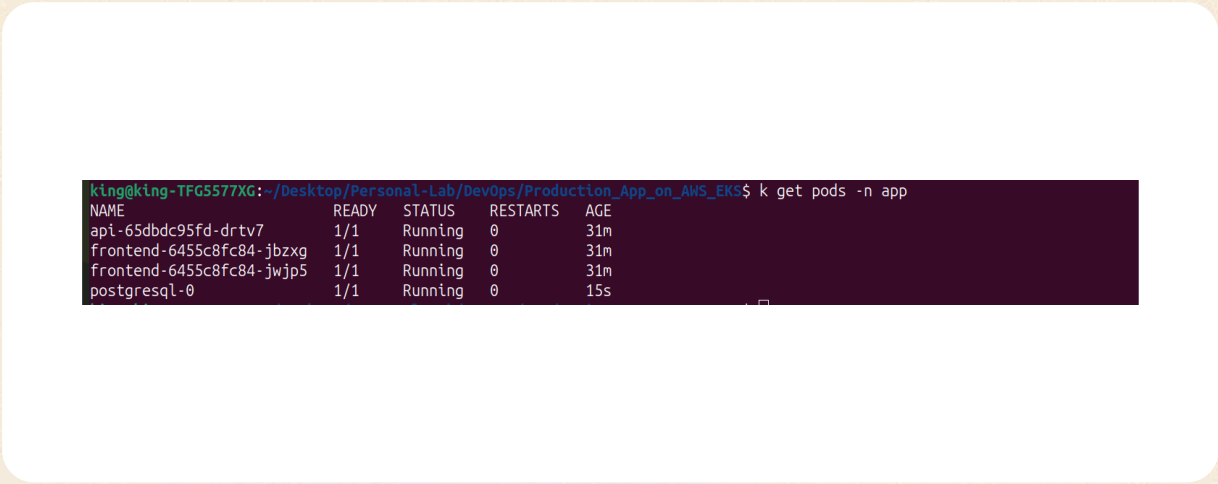
Why least-privilege RBAC matters

Namespaces isolate resources within a cluster to prevent collisions, and restricted Roles ensure that users or groups under that role have the exact permissions they need to execute their tasks within the cluster.

Deploying the Multi-Tier Application with Helm

Helm chart design and deployment

In this step, I'm setting up a custom Helm chart, so that I can package a multi-tier application:an nginx frontend, an httpbin API, and a PostgreSQL database with persistent storage.



ALB provisioning and Ingress routing

My ALB DNS name is "app.kahmedt.com". The Ingress routes / to the frontend and /api to the httpbin api service. This is all done through host routing and path-based routing.

Configuring Health Checks and Zero-Downtime Rolling Updates

Probe and rollout strategy

In this step, I'm configuring healthchecks for my pods so that my deployments can easily be restarted if any pods are found to be unhealthy or not ready to receive traffic.

Liveness vs readiness probes

A liveness probe checks whether the container is still alive, while a readiness probe checks whether the container is ready to receive traffic or not. I used different paths because both the nginx and httpbin apps listen and respond to requests on different endpoints.

Automating Scaling with the Horizontal Pod Autoscaler

HPA setup and configuration

In this step, I'm setting up an HPA so that more pods can be deployed when traffic spikes up.

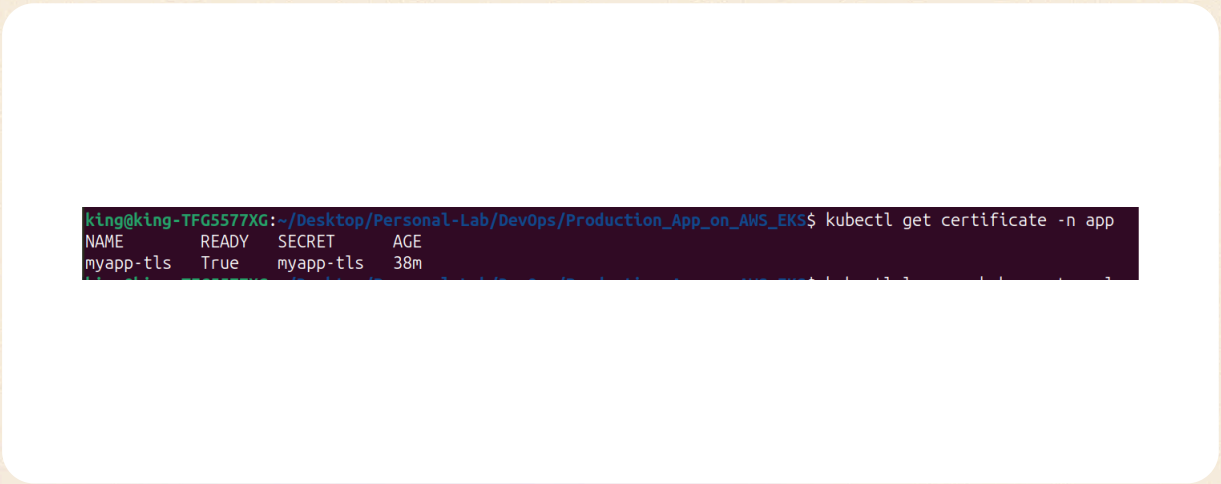
How the Metrics Server enables autoscaling

The Metrics Server provides the metrics(CPU or Memory utilization) which the HPA uses to make a make decision on whether to scale the number of pods or not

Automating DNS and TLS with External-DNS and cert-manager

External-DNS and cert-manager setup

In this step, I'm setting up an External-DNS and cert-manager so that my application can have an easier, secure and and established way of comunicating with clients.



DNS-01 challenge validation explained

DNS-01 is preferred because it works before your domain resolves to the cluster. There is no chicken-and-egg problem with ALB provisioning. It also supports wildcard certificates if you need them later.

Full Observability with Prometheus and Grafana

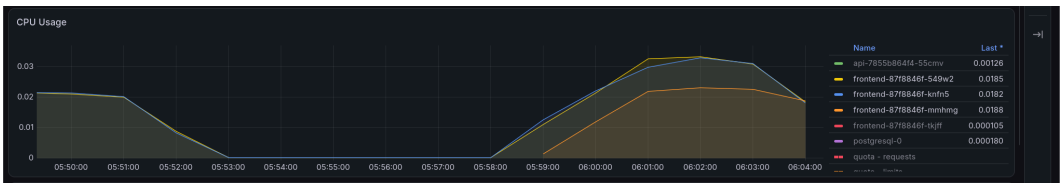
Monitoring stack deployment

In this step, I'm setting up monitoring using kube-prometheus-stack, so that I can observe the happenings in my cluster through a dashboard.

Grafana configuration and persistence decisions

The default credentials are "admin" and "prom-operator" for the username and password, but the password may sometime be different due to changes in chart settings during Helm release upgrades, integration of external secrets, or Grafana. Persistence is disabled because this project does not require data metrics data or dashboards info to be saved.

Load Testing and Watching Autoscaling in Real-Time



HPA scaling behavior under load

In this project extension, I observed the HPA scaling from 2 to 3 replicas when CPU exceeded 50% utilisation.

Reflections and Key Takeaways

Tools and concepts mastered

The key tools I used include Amazon EKS, Route53, IAM, ACM and an ALB. Key concepts I learnt include how to provision a 3-tier production grade app on multiple nodes with IRSA, EBS CSI Driver for persistent storage, RBAC policies, an AWS Load Balancer Controller for ALB-based ingress, TLS certificates with cert-manager and ACM, deployed a full Prometheus + Grafana observability stack, and Load-tested the application and watched HPA scale pods in real-time through Grafana dashboards

Time and challenges

This project took me approximately 6 hours(with research and error troubleshooting included). The most challenging part performing the load testing to verify actual scaling of the pods based based on CPU usage.

Looking ahead

I did this project today to learn how to experience how apps are deployed in production with Amazon EKS. Another skill I want to learn is.